What Actually Improves Low-Resource ASR: Lessons from Thirty-Seven LoRA Adaptations Across Eight Border Languages

Anonymous Author(s)

Abstract

We report the adaptation campaign behind a deployed, fully offline speech-recognition system for eight low-resource border languages: 37 LoRA fine-tuning runs across two multilingual backbones. Its headline outcome is a complete backend replacement—per-language fine-tuned Whisper, the system's original design, now serves *none* of the eight languages—but its more transferable results concern two languages absent from both backbones' language inventories, where the failures turn out to be representational rather than acoustic.

A language a model cannot name is not rejected but silently misrouted: given Dogri audio, Whisper's language identification answers *Punjabi* for 222 of 425 clips and emits Gurmukhi against Devanagari references, yielding a word error rate above 100% that no accuracy dashboard would flag as a missing-language problem. Adding the missing language token, initialised from a neighbour and trained, takes Dogri from 102.39% to 46.73% WER—the largest single-language gain of the campaign, in a language never previously measured. The same technique on Kashmiri exposed a subtler defect: 20 characters of written Kashmiri have no token in the backbone's subword vocabulary and resolve to the unknown-token id, appearing in 96.9% of training sentences; of the 747 test words containing the affected letters, the pre-repair model got *all 747* wrong, and after repair it recovers 310. Finally, a retrospective paired bootstrap divides the campaign sharply: backend replacement, vocabulary repair and training to convergence hold at $p < 0.001$, whereas every adapter-versus-adapter decision—including one on which a deployment was made—is indistinguishable from noise on the 30- and 100-clip evaluations used to gate them. We close by measuring the deployed code path directly and finding it 2.07 pp worse than the path that produced every number we had published.

CCS Concepts

• **Computing methodologies** → **Speech recognition**; *Natural language processing*.

Keywords

speech recognition, low-resource languages, parameter-efficient fine-tuning, deployment, evaluation methodology

1 Introduction

Most practical work on low-resource speech recognition assumes the target language is at least *representable* by the model being adapted—that it has a language token, and that its script survives the subword vocabulary intact. For the languages that motivate this paper, neither assumption holds, and the consequences are not the ones the literature prepares you for.

We describe the adaptation campaign behind a deployed offline speech pipeline for eight border-region languages: Punjabi, Pashto, Urdu, Nepali, Mandarin, Hindi, Kashmiri and Dogri. The system runs air-gapped on a single workstation, transcribing degraded radio audio and translating it to English; its original design fine-tuned one Whisper model per language. Over 37 adaptation runs we replaced that design entirely, accumulating results we believe are more useful to practitioners than the final error rates: several things that obviously should have helped did not, one that looked like a modelling problem was a vocabulary bug, and a retrospective audit showed that many of the decisions we made along the way were unsupported by the evaluations used to make them.

Contributions.

- A backend-selection study across eight languages showing that one shared multilingual model with per-language LoRA adapters beats per-language fine-tuned Whisper everywhere, in clean and degraded audio alike, once three scoring defects in the original comparison are corrected (Section 4); and a per-run account of all 37 adaptations, including the failures (Section 5).
- A method for languages the backbone cannot name—adding a language token initialised from a linguistic neighbour and training it—demonstrated on two languages with different scripts and failure profiles. We show by measurement that the correct neighbour is determined by *script*, not genetic relatedness (Section 6).
- The identification and repair of a subword-vocabulary defect that placed a hard floor under Kashmiri accuracy: 20 characters resolving to the unknown token across 96.9% of training sentences, rendering 747 of 747 affected test words unwinnable regardless of capacity or training duration (Section 7).
- A retrospective significance audit of every decision the campaign took, showing that our 30-clip robustness gate could not resolve the differences it was used to decide (Section 8); and a direct measurement of the deployed code path against the evaluation harness, which differ by 2.07 pp (Section 9).

This is an experience report, and much of its value lies in the negative results: we deployed an adapter on a 0.75 pp margin we later found insignificant, rejected two others on differences that were also insignificant, and spent a day on a decoding improvement that traded clean-speech accuracy for robustness at low signal-to-noise ratio—the wrong direction for a radio system. Each is reported with the evidence that undermines it. The conditions that produced them—small evaluation sets, sub-2 pp effects, a gate inherited from an earlier phase—are entirely ordinary.

2 Related Work

Whisper [1] and SeamlessM4T [2] are the backbones we adapt; MMS [7] extends coverage past 1,000 languages and is the standard answer to "my language is not in the model". We deliberately did not take that route: the deployment constraint is a single 8 GB GPU already hosting recognition, language identification, translation and summarisation, and a second acoustic backbone for two languages was not affordable. This paper asks what is achievable *without* it, by repairing the representation of a backbone already resident in memory.

Parameter-efficient adaptation via LoRA [3] is routine for speech and its mechanics are not our contribution; what we add is a per-decision audit of a campaign that used it—which of 37 runs produced effects that survive a paired bootstrap [9], and which did not. Likewise, extending a pre-trained model to an unseen language by adding and training embedding rows is established in machine translation and multilingual language modelling. Our contribution there is narrower and empirical: for a *decoder-conditioning* token the initialisation neighbour must be chosen by script rather than language family, the choice is worth 14.8 pp WER, and leaving the added row frozen—the natural reading of "initialise from a neighbour"—costs a further 21 pp. We also quantify a failure mode we have not seen reported: a subword vocabulary that silently maps a fifth of a language's distinctive characters to the unknown token, invisible to loss curves and aggregate WER alike. Evaluation resources are thin: FLEURS [4] covers six of the eight, while Kashmiri and Dogri appear in neither FLEURS nor Common Voice, so we use IndicVoices-R [5], whose Dogri split we believe carries the first published ASR numbers for that language.

3 Setting

The system transcribes intercepted radio traffic offline. Every model runs locally on one workstation with an 8 GB consumer GPU; no audio leaves the machine, which rules out hosted APIs and constrains model size throughout. Audio arrives degraded—narrowband, noisy, codec-compressed—so clean-speech accuracy alone is never a sufficient basis for a deployment decision, a point that recurs throughout Section 8.

Languages and data. Six of the eight languages have FLEURS test splits, used at $n = 100$ each. Kashmiri and Dogri do not appear in FLEURS at all; for these we use the IndicVoices-R test splits, 372 and 425 clips respectively. This asymmetry matters more than it first appears: Section 8 shows the 100-clip splits cannot resolve the differences the campaign spent most of its effort chasing.

Scoring. Four of the eight languages use Perso-Arabic or Devanagari scripts whose orthographic conventions make raw word error rate misleading. We therefore score on a normalisation ladder: raw text (L0), Unicode NFC and whitespace normalisation (L1), diacritic stripping (L2), and conservative and aggressive character folding (L3, L4). **We report L2 throughout**, including for figures published elsewhere at L0. The choice is consequential for Kashmiri, where L0 and L2 differ by roughly 14 pp because Perso-Arabic references are densely diacritised and no model reproduces those marks; it is nearly irrelevant for Dogri, where L1 through L4 are identical. One scorer is used for every language and system, including character segmentation for Mandarin.

Table 1: Deployed word error rate by language, before and after the campaign. Six languages are scored on FLEURS ($n = 100$); Kashmiri and Dogri on IndicVoices-R ($n = 372$, $n = 425$) at L2. "Before" is the previously deployed per-language fine-tuned Whisper, except Dogri, which had no model and fell back to un-fine-tuned Whisper.

Language	Script	Before	After	Deployed backend
Punjabi	Gurmukhi	57.39	**19.77**	SM4T zero-shot
Nepali	Devanagari	50.92	**24.34**	SM4T + LoRA
Hindi	Devanagari	19.78	**12.91**	SM4T + LoRA
Urdu	Nastaliq	19.82	**16.90**	SM4T zero-shot
Mandarin	Han	14.22	**11.69**	SM4T zero-shot
Pashto	Nastaliq	38.55	**36.16**	SM4T + LoRA
Kashmiri	Nastaliq	65.19	**50.26**	SM4T + custom token
Dogri	Devanagari	102.39	**46.73**	SM4T + custom token

Adaptation. All adaptation is LoRA. The 11 Whisper large-v3 runs used rank 8–16 and consumed roughly 100 GPU-hours on the workstation; the 26 SeamlessM4T adapters ranged from rank 8 to 128 on the attention and MLP projections, the upper end requiring rented cloud GPUs because rank 128—6.6% of model parameters, about 107 M, against rank 32's 1.75%—exceeds 8 GB. Total cloud expenditure was approximately $25.

4 Backend Replacement

The system originally fine-tuned one Whisper large-v3 model per language. A re-evaluation against zero-shot SeamlessM4T v2 reversed that design completely.

Table 1 and Figure 1 give the outcome. Fine-tuned Whisper now serves none of the eight languages; every model remains on disk for rollback but none is in the serving path. For five languages, zero-shot SeamlessM4T won outright, by margins from 2.9 pp (Urdu) to 37.6 pp (Punjabi). The remaining three required adapters, and two of those required more than an adapter.

The advantage widens under degradation. A backend chosen on clean speech would be the wrong artefact to ship here, so we repeated the comparison across five channel conditions—clean, bandpass-filtered, additive noise at 10 dB and 0 dB SNR, and 16 kbps MP3—at $n = 30$ per cell. Figure 2 shows SeamlessM4T ahead in all 25 cells, with its margin largest where the channel is worst: for Hindi the advantage grows from 5.0 pp clean to 19.5 pp at 0 dB, for Mandarin from 3.6 to 17.2 pp. The decision would have been the same either way, but a clean-only comparison would have understated the win.

Three scoring defects preceded the result. The original comparison favoured Whisper, and the reversal came from correcting the measurement, not the models. First, the "untuned baseline" loaded `whisper-large-v3-turbo` rather than `large-v3`, a different model with different behaviour. Second, Mandarin WER was computed without character segmentation, so unspaced Han transcripts scored near 100% regardless of content—an artefact reported

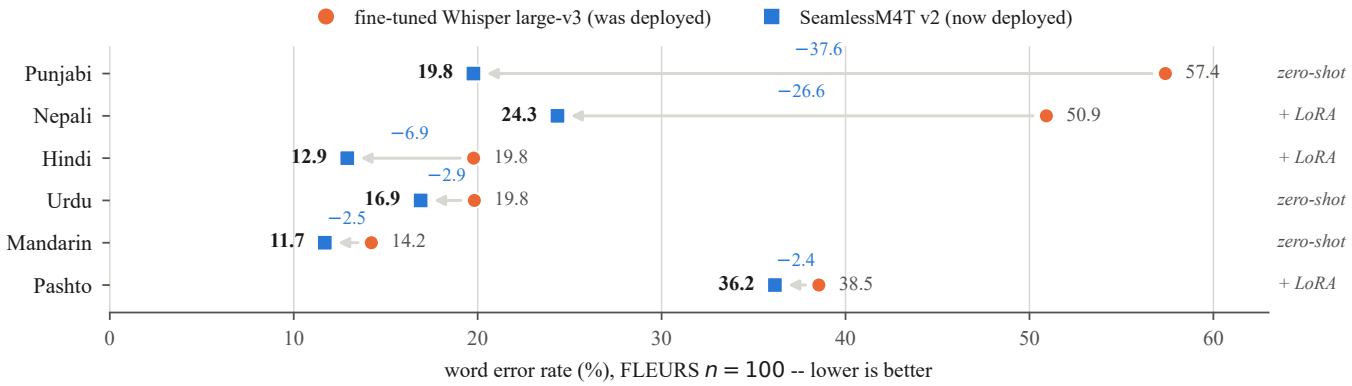

Figure 1: Backend replacement on the six FLEURS languages. Every language moves left. Three of the six needed no adaptation at all—zero-shot SeamlessM4T already beat a Whisper model that had been fine-tuned on that language.

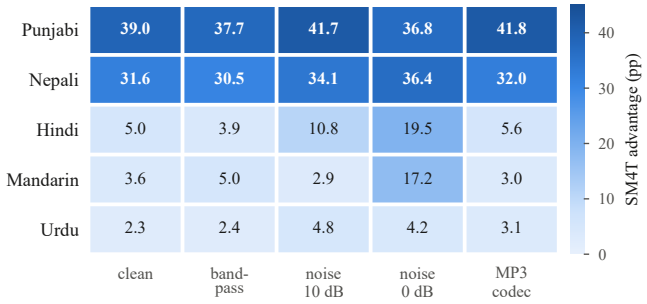

Figure 2: SeamlessM4T's word-error-rate advantage over fine-tuned Whisper, in percentage points, across five channel conditions ($n = 30$ per cell). Positive everywhere; largest under additive noise.

at the time as a headline improvement from 100% to 9%. Third, a label-encoding bug in the speech-to-text-translation path depressed one backend's scores independently of its recognition quality; fixing it moved Pashto translation chrF from 2.12 to 37.60 while leaving recognition untouched, which is what identified it as a scoring rather than a modelling fault. Each defect had produced a plausible number that survived review because nothing about it looked wrong. This establishes the pattern the rest of the paper documents: in this project, most apparent model findings we investigated closely turned out to be measurement findings.

5 The Adaptation Campaign

Table 2 lists the runs on which a decision turned, and Figure 3 plots the two languages that took the most work. The levers divide cleanly into those that moved word error rate and those that did not.

Pashto: seven attempts, one real gain. Pashto was fine-tuned Whisper's last stronghold and took the most effort. Adding Common Voice data to FLEURS in a mixture dominated by the former *regressed* accuracy by 5.6 pp ($p < 0.001$), a domain-drift effect; rebalancing the mixture and raising adapter rank recovered it. The decisive change was neither data nor capacity but *noise-augmented training*: degrading the training audio with the same bandpass, additive-noise and codec pipeline used for evaluation produced an adapter that improved clean speech and, more importantly, stopped collapsing at low signal-to-noise ratio. Two later attempts—tripling the Common Voice share, and raising rank to 128—produced changes we cannot distinguish from noise (Section 8). The lesson the campaign eventually drew is that more data was harmful when it *unbalanced* the mixture, not simply because there was more of it.

Training to convergence is worth more than it appears. Two runs stopped not because they had converged but because their stopping criterion expired—one an early-stopping patience too tight for a corpus of that size, the other a linear learning-rate schedule decayed to zero. Both recorded their best validation loss at their final step, having improved at essentially every evaluation. Restarting each from its own weights with a fresh schedule, changing nothing else, improved Kashmiri by 3.84 pp and Dogri by 3.34 pp, both at $p < 0.001$. The two languages differ in script, corpus and backbone history, and the effect replicates at almost the same magnitude. A schedule that expires produces a loss curve indistinguishable from convergence, and mistaking one for the other cost this campaign more than doubling adapter rank gained it—at roughly $5 of rented GPU time per rerun, it was also the cheapest fix available.

What the Whisper phase cost, and what it taught. Panel (a) of Table 2 is, in hindsight, 100 GPU-hours on the wrong backbone; every model in it has been rolled back. It still produced two diagnostics we relied on afterwards. First, per-checkpoint WER oscillates while training loss falls monotonically—Punjabi v3 regressed 1.2 pp between steps 1200 and 1400 while its loss improved, then recovered 2.7 pp by step 1600—so early stopping on WER would repeatedly have halted a still-improving run, which is what later made us suspicious of the two expired schedules. Second, evaluation rather than training dominated wall-clock: with `predict_with_generate` at batch size 1, one Punjabi v3 checkpoint evaluation took 2.5 hours, so 20 evaluations added 50 hours to a 22-hour training run. The Mandarin run is a third kind of lesson: it diverged at step 820 under fp16 gradient overflow and its best checkpoint still lost to

Table 2: The adaptation campaign. Panel (a), the Whisper phase, is scored on the training-split validation set—the quantity that selected checkpoints at the time; Table 1 gives the corresponding held-out figures. Panel (b), the SeamlessM4T phase, is scored on held-out test data: FLEURS $n = 100$ for Pashto, IndicVoices-R $n = 372/425$ at L2 for Kashmiri and Dogri. Omitted are the single-run adaptations for Urdu, Hindi and Nepali and two runs superseded by the label-encoding fix. "n.s." marks a change that does not survive the bootstrap of Section 8.

Run	Lang.	Rank	Data	Steps	WER	Δ	Outcome
(a) Whisper large-v3, per-language fine-tuning — 11 runs, ≈100 GPU-h on the workstation							
pa v1	Punjabi	8/16	FLEURS	3,000	56.67	—	superseded
pa v2	Punjabi	8/16	+ IndicVoices-R	3,000	52.55	−4.1	superseded
pa v3	Punjabi	16/32	+ IndicVoices-R 20k	4,000	49.31	−3.2	deployed, later rolled back
ne v1	Nepali	8/16	FLEURS	2,000	52.14	—	superseded
ne v2	Nepali	8/16	+ IndicVoices-R	3,000	50.82	−1.3	deployed, later rolled back
zh v1	Mandarin	8/16	FLEURS	400	8.97	—	**diverged; never served**
ps v1	Pashto	8/16	FLEURS	2,000	38.55	—	deployed, later rolled back
ps_lv3	Pashto	16/32	FLEURS + Common Voice	3,000	50.91	+12.4	**rejected**
ur v1	Urdu	8/16	FLEURS	1,000	19.82	—	deployed, later rolled back
hi v1	Hindi	8/16	FLEURS	600	19.78	—	deployed, later rolled back
ks v1	Kashmiri	8/16	IV-R + custom <|ks|>	3,000	74.02	—	deployed, later rolled back
(b) SeamlessM4T v2, LoRA adapters — 26 runs; the 17 on which something turned							
ps	Pashto	8/16	FLEURS	2,000	41.30	—	superseded
ps_cv	Pashto	8/16	+ Common Voice (dominant)	3,000	42.47	+1.2	**rejected: domain drift**
ps_bal	Pashto	16/32	rebalanced mixture	3,000	39.72	−2.8	superseded
ps_bal2	Pashto	32/64	+ MLP projections	3,000	37.29	−2.4	superseded
ps_aug	Pashto	32/64	+ noise-augmented training	3,000	36.91	−0.4	superseded
ps_aug2	Pashto	32/64	3× Common Voice share	3,000	37.46	+0.5	rejected (n.s.)
ps_cloud	Pashto	128/256	as ps_aug, rank 128	3,000	**36.16**	−0.8	**deployed** (n.s.)
ks	Kashmiri	8/16	IV-R 20k, token *frozen*	3,000	85.25	—	unusable
ks_r16	Kashmiri	16/32	as above, larger rank	3,000	79.38	−5.9	unusable
ks_max	Kashmiri	32/64	token *trainable*	6,000	64.31	−15.1	deployed, superseded
ks_max2	Kashmiri	32/64	97k clips / 240 h	8,000	61.88	−2.4	superseded
ks_cloud	Kashmiri	128/256	145k clips / 335 h	6,000	56.44	−5.4	superseded
ks_cloud2	Kashmiri	128/256	as above, trained to convergence	12,000	52.60	−3.8	superseded
ks_cloud3	Kashmiri	128/256	+ 20 repaired vocabulary chars	12,000	**50.26**	−2.3	**deployed**
ks_cloud4	Kashmiri	128/256	warm start, token rows at 5× LR	1,500	50.69	+0.4	rejected (n.s.)
doi_iv	Dogri	128/256	IV-R Dogri, custom __doi__	6,000	50.07	—	schedule expired
doi_iv2	Dogri	128/256	as above, fresh schedule	9,000	**46.73**	−3.3	best Dogri model

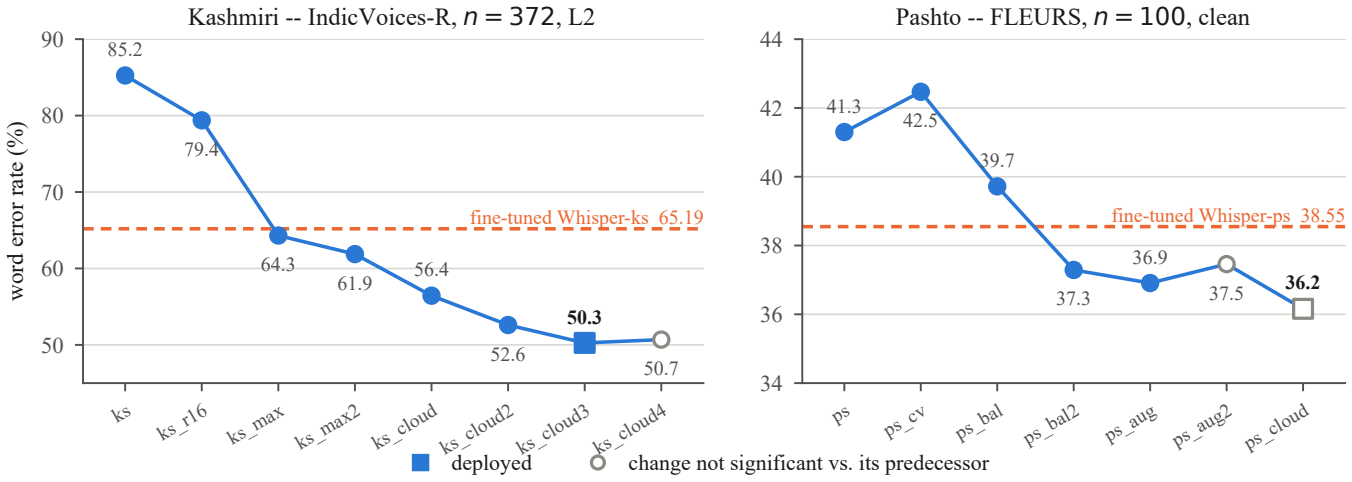

Figure 3: The two languages that took the most work, every run on one scorer. Kashmiri's decisive moves are making the added language token trainable (`ks_max`) and repairing the subword vocabulary (`ks_cloud3`); Pashto's is noise-augmented training (`ps_aug`). Hollow markers are runs whose change from their predecessor does not survive a paired bootstrap—including the one we deployed.

the un-fine-tuned baseline, so fine-tuning was abandoned for that language—correctly, as zero-shot SeamlessM4T later beat both.

6 Languages the Model Cannot Name

Kashmiri and Dogri are absent from both backbones' language inventories. SeamlessM4T v2 ships 101 language tokens, including `__hin__`, `__pan__` and `__urd__`, but neither `__kas__` nor `__doi__`; Whisper has no `<|ks|>` or `<|doi|>`.

The failure is silent. A model given a language it cannot name does not refuse; it routes the input to whatever it considers nearest. Presented with Dogri audio, Whisper's language identification answers *Punjabi* for 222 of 425 clips against 25 for Hindi—a linguistically defensible judgement, since Dogri is far closer to Punjabi—then transcribes in **Gurmukhi**, while Dogri is written in Devanagari. Word error rate exceeds 100% because insertions outnumber the words that could have been matched. Nothing in the pipeline reports an error; the system produces confident, fluent, entirely unusable output, and did so for the entire period Dogri was nominally supported.

Method. We add the missing token to the tokeniser, initialise its embedding from the closest supported language, and—critically—make that embedding trainable, using PEFT's trainable-token mechanism [6] to learn a delta on that row alone while the rest of the embedding table stays frozen. The first two Kashmiri attempts left the row frozen, treating it as a fixed conditioning vector, and reached only 85.25% and 79.38% WER—worse than the Whisper model they were meant to replace. Unfreezing that single row, with rank raised at the same time, took the next run to 64.31% (Figure 3).

Which neighbour? Script, not ancestry. The initialisation source is a real design decision, and the intuitive answer is wrong. Dogri's nearest relative among the supported languages is Punjabi, and Whisper's language identification agrees emphatically. But SeamlessM4T knows Punjabi only in Gurmukhi, and a language token conditions *generation*, not recognition. Figure 4 settles it: zero-shot decoding with the Punjabi token scores 114.62% WER against 99.86% for Hindi, and the character error rates—96.79% versus 67.99%—show why. The Punjabi prior emits the wrong script, so almost nothing matches. Recognition and generation want different neighbours; the token serves generation, so script wins. We initialised `__doi__` from `__hin__` on this evidence, and `__kas__` from `__urd__` on the same reasoning.

The result is a 55.7 pp improvement over what the deployed system actually did, and 41.5 pp over the best baseline obtainable without training—the largest single-language gain of the campaign, from a language nobody had measured, using a technique already in the codebase for another one. That the same method works unchanged on two languages with different scripts and different failure profiles is our basis for calling it a method rather than a fix.

7 A Floor Made of Missing Characters

Applying the technique to Kashmiri exposed a second defect, invisible to every diagnostic we routinely used.

After training a rank-128 Kashmiri adapter to 56.44% WER, we examined the composition of the remaining errors rather than

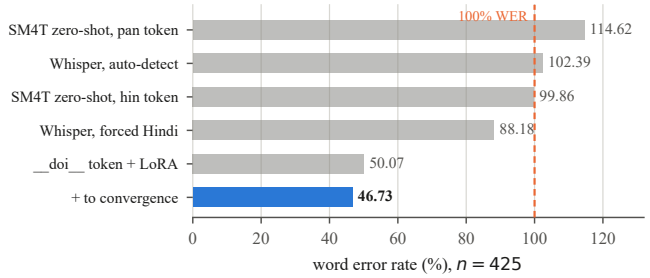

Figure 4: Dogri, 425-clip IndicVoices-R test split. The two zero-shot proxies also answer the initialisation question: the script-matched neighbour beats the genetically closer one by 14.8 pp WER and 28.8 pp CER. Everything above the dashed line is a system producing more errors than there are reference words. All five figures are L2 on the same 425 clips and the same scorer.

the aggregate. Substitutions accounted for 77%, the hypothesis-to-reference length ratio was 0.943, and no output was truncated—so the early-termination pathology that had afflicted earlier Kashmiri models was gone, and little was recoverable by adjusting decoding. The errors were words the model wrote incorrectly, not words it failed to write. The most frequent confusions were pairs differing by a single character, and those characters were Kashmiri-specific vowel signs. Testing them against the tokeniser gave the explanation: they encode to the **unknown token**.

Extent. Twenty characters used in written Kashmiri have no token in SeamlessM4T's subword vocabulary. They occur 854,234 times in the training corpus and appear in **96.9%** of its sentences. Two consequences follow. The model cannot emit them—one such character appears 370 times in the test references and *zero* times in any hypothesis. And because the same tokeniser encodes the training targets, the model was trained against corrupted labels for nearly every sentence it saw.

Cost. Of the 3,917 substitution errors, 662 fall on words rendered unrepresentable by these characters. Thirteen of the twenty are combining marks that diacritic normalisation removes before scoring; the seven that survive it include five *letters*, four of which occur in this test set. Those four appear in 747 of its 8,988 word tokens—8.3%, across 211 distinct word types—and the pre-repair model got **747 wrong. Every one.** No amount of capacity, data or training time could have fixed a single one of them.

Repair and result. We add all twenty characters as tokens, initialise each from its closest in-vocabulary neighbour (letters from their base letter, combining marks from a similar mark) and train those rows alongside the language token. Nothing else changes—same corpus, rank, schedule and converged step—so the comparison isolates a single variable. Word error rate falls from 52.60% to **50.26%** ($p < 0.001$), and of the 747 previously unwinnable words the repaired model recovers **310**; the remaining failures are ordinary acoustic confusions rather than blank omissions.

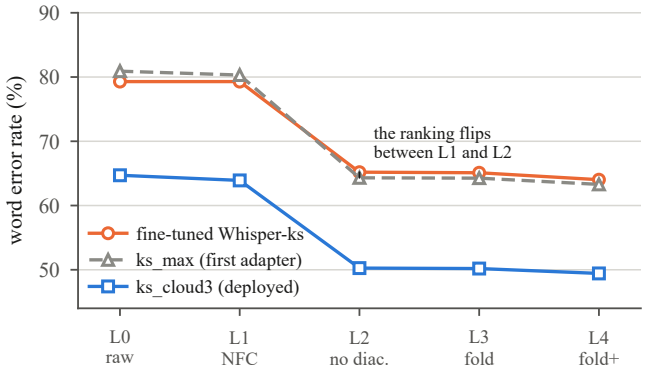

Figure 5: The same three Kashmiri systems on all five rungs of the normalisation ladder (n = 372). The choice of rung is worth more than most of the modelling decisions in this paper, and between L1 and L2 it reverses which of two systems appears better—the error that once kept a fine-tuned Whisper model in production.

Why the aggregate gain is smaller than the defect. Figure 5 shows the repair on all five rungs of the normalisation ladder. On unnormalised text it is worth 9.60 pp (74.31% to 64.71%); at L2, the level we decide on, 2.34 pp. The reason is that the diacritic filter and the defect overlap: 13 of the 20 repaired characters are combining marks that L2 deletes before scoring, so the deciding ruler sees the repair almost entirely through the four letters. Both numbers are real, and reporting only the larger would misrepresent what the repair buys under the metric we decide on.

Generality. This defect is not exotic. It arises whenever a language's written form uses characters outside a subword vocabulary trained predominantly on other languages—the ordinary condition for scripts with language-specific extensions. It is invisible to loss curves, because the model learns to predict the unknown token perfectly well; invisible to aggregate WER, because affected words appear as ordinary errors; and invisible to any evaluation that does not inspect which characters the model can produce. Dogri, written in unextended Devanagari, shows *zero* unsupported characters, which is why it needed only the language token.

8 What the Evaluation Could Not See

Having reached deployable accuracy on all eight languages, we ran a retrospective audit: for every comparison on which a decision had been taken, a paired bootstrap over clips (10,000 resamples, re-aggregating errors and reference length rather than averaging per-clip rates). Pairing matters, since both systems transcribe identical audio and removing clip-difficulty variance is what makes small effects detectable at all. The audit divided the campaign along a line we had not anticipated (Figure 6).

The split follows test-set size and effect size together. Every conclusion about *backend selection*—the question the project originally asked—is supported, because replacing one model family with another moves word error rate by 10 to 22 pp and even 30 clips resolve that comfortably; so are the two mechanisms established on the 372- and 425-clip Kashmiri and Dogri sets. What does not survive is the fine-grained adapter selection that consumed most of the campaign's effort. The 1 to 3 pp differences between an adapter and its successor, evaluated on a 30-clip robustness sweep and a 100-clip clean split, are indistinguishable from noise in every case we tested—including the 0.75 pp margin on which we deployed one adapter over another.

"Wins 4 of 5 conditions" is not a result. Our deployment gate reported robustness as a win count across five degradation conditions, a format that reads like evidence. Decomposed, the four wins are four differences whose confidence intervals all cross zero, and the count itself has a one-sided sign test $p = 0.188$. The gate was designed during earlier work when the differences under test were tens of points—as in Figure 2, where every cell is a 2 to 42 pp gap—and was never revisited as those differences shrank to fractions of one. A gate adequate for choosing a backbone is not automatically adequate for choosing between two checkpoints of the same model.

We did not reverse the deployments: choosing the newer of two statistically indistinguishable adapters is defensible under uncertainty, and one does not replace a running model without evidence of improvement. What changed is the claim—those adapters are *equivalent on the available evidence*, not better. We report it because the alternative, quietly restating point estimates as findings, is precisely the practice that produced three of this paper's corrections.

The same blindness in the decoder. The gate is not the only place a clean-speech number misleads. Late in the campaign we found that the deployed system decodes greedily—the backbone ships no beam-search default and the serving code never sets one, so all eight languages had been decoding with a single beam without anyone choosing it. Beam search with width 8 improves Kashmiri clean WER from 50.26% to 47.37% on the 372-clip split, and a Kneser–Ney trigram [8] rescoring the 8-best list reaches 46.90%. We did not deploy either. Across seven beam-and-length-penalty configurations, *every one* regressed the 0 dB condition, by 1.99 to 3.15 pp; width 2 managed to lose 0.50 pp on clean audio as well, making it worse in both at once. With flat posteriors, beam search reliably finds higher-scoring hypotheses that are fluent and wrong, and greedy decoding's myopia is a virtue exactly when the acoustic evidence is weakest. **Decode settings validated only on clean test sets can silently degrade deployed robustness**—and the clean-speech number, the one a paper would normally report, is the one that moves in the flattering direction.

9 The Deployed Path Is Not the Evaluated Path

A final measurement closes the paper because it undercuts the rest of it. Every word error rate we have reported was produced by our evaluation harness, which constructs the model with the adapter loaded through the parameter-efficient fine-tuning library. The deployed system constructs it differently: it bakes the trainable-token embedding deltas into the base model and loads only the low-rank matrices, because the library cannot stack a trainable-token delta alongside several named adapters. The two constructions were assumed equivalent for a year and never compared.

They are not. Measured on the same 372 clips with the same scorer, Kashmiri scores **52.33%** through the deployed path against **50.26%** through the evaluation path: a 2.07 pp gap, 95% CI [+1.24, +2.91],

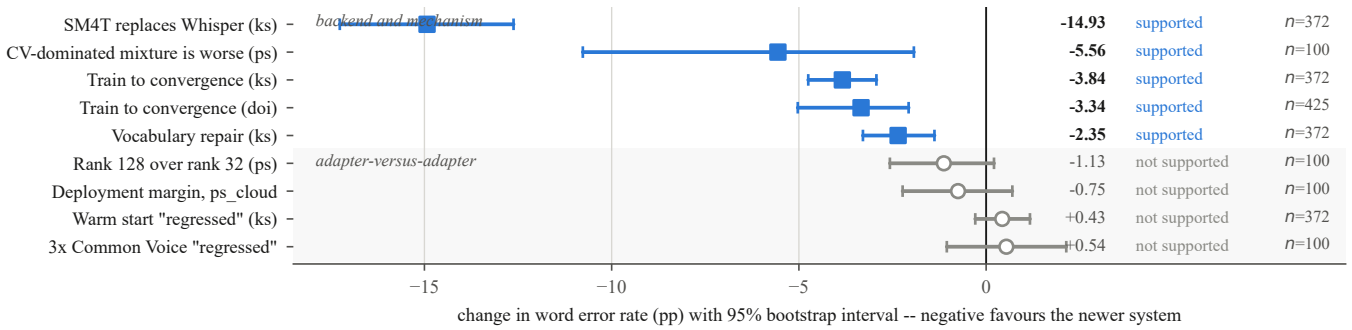

Figure 6: Retrospective paired bootstrap over every claim on which a deployment or a rejection was decided. The split is not between good and bad ideas—it is between questions the available test sets were large enough to answer and questions they were not.

$p < 0.001$, with only 19 of 372 hypotheses identical between them. The deployed model is measurably worse than the model this paper describes, and because the divergence lives in the shared generation path rather than in anything language-specific, the other seven languages are presumed affected and have simply never been measured both ways.

Investigating it fixed one genuine defect—the deployment baked only the first of 21 trainable-token deltas, leaving the twenty repaired characters of Section 7 at their initialisation values in production. The residual 2 pp is unresolved: direct comparison has excluded the low-rank weights (480 of 480 tensors identical), the deltas, the extracted input features, the generation length limit, the input dtype, substituting the checkpoint's frozen embedding table, and multi-adapter interference. We report it rather than correcting it silently—republishing against the deployed path would mean re-evaluating eight languages, and the gap is in any case the clearest instance of what this paper keeps finding: the artefact you measure is not necessarily the artefact you ship.

10 Lessons

(1) **Check what your model can emit before adapting it.** A tokeniser audit costs minutes and would have found the 20-character defect a month earlier. Loss curves, aggregate WER and validation accuracy are blind to it.

(2) **A missing language fails silently**, routing to its nearest neighbour and producing confident, well-formed, useless output. Verify every language you claim to support is one the model can name—and that the added token is *trainable*, worth 15 pp here. Initialise it **by script, not by ancestry**: the token conditions generation, so a genetically closer language in the wrong script starts you 14.8 pp further away.

(3) **A schedule that expires is not a converged model.** Two runs here left 3–4 pp on the table by stopping when their stopping criterion, rather than their loss, ran out. Both reruns cost about $5.

(4) **Size the evaluation to the effect, and validate on the channel you deploy to.** A 30-clip sweep resolves backend replacement and nothing finer; if you are chasing 1 pp, report intervals or stop claiming improvements. And every beam configuration we tried bought clean-speech accuracy with robustness at 0 dB, which a clean-only test set would have called a win.

(5) **Measure the deployed path.** Evaluation and serving code diverge quietly, and the divergence is worth points.

11 Conclusion

We replaced a per-language fine-tuned Whisper deployment with a single shared multilingual model and per-language adapters across eight low-resource border languages, including two the backbone cannot name and one never measured at all. The largest gains came not from capacity or data but from repairs to representation: a language token the model lacked, and twenty characters its vocabulary could not express. The most useful result, though, may be the audit: of the decisions this campaign made, those about backends were supported by the evidence and those about individual adapters largely were not, and we could not have known which was which without asking.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023.
[2] Seamless Communication et al. SeamlessM4T: Massively multilingual and multimodal machine translation. arXiv:2308.11596, 2023.
[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
[4] A. Conneau, M. Ma, S. Khanuja, Y. Zhang, V. Axelrod, S. Dalmia, J. Riesa, C. Rivera, and A. Bapna. FLEURS: Few-shot learning evaluation of universal representations of speech. In *IEEE SLT*, 2023.
[5] T. Javed et al. IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages. In *ACL Findings*, 2024.
[6] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.
[7] V. Pratap, A. Tjandra, B. Shi, et al. Scaling speech technology to 1,000+ languages. *JMLR*, 25(97), 2024.
[8] K. Heafield. KenLM: Faster and smaller language model queries. In *WMT*, 2011.
[9] M. Bisani and H. Ney. Bootstrap estimates for confidence intervals in ASR performance evaluation. In *ICASSP*, 2004.